

SurakshaNet: on-device abuse detection for Hinglish chat

B.Tech mid-semester briefing · 15 minutes

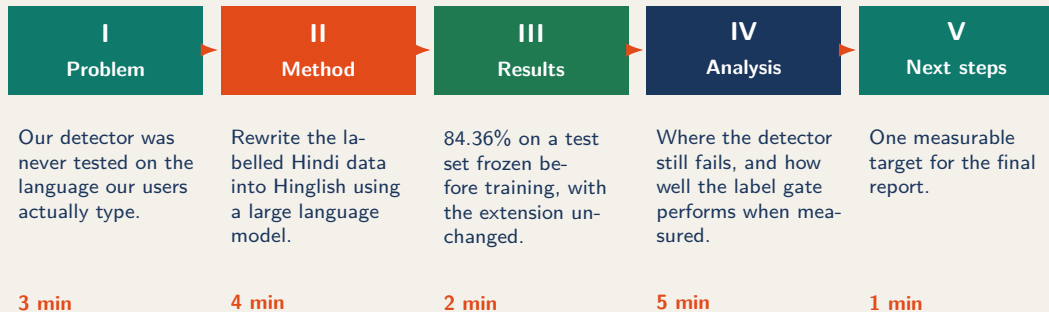
Aamod Jain and Pratham Garg · 20 September 2026

WHAT SHIPS: A BROWSER EXTENSION, NOTHING ELSE



No message ever leaves the device. The contribution is training data, not a new inference path.

What this presentation covers



Part IV is the substance of this iteration. Having established a working Hinglish detector in Part III, the effort since then has gone into measuring *where* it fails and *why*, which is what sets the agenda for the final report.

Six terms used throughout this presentation

Hinglish

Hindi grammar, written in Roman letters, with English words mixed in. This is what people actually type.

Code-mixing index (CMI)

How deeply two languages are mixed in one sentence. 0 = only one language. Humans average ≈ 36 .

MACD

Our labelled dataset: Hindi comments in Devanagari script. Label 0 = abusive, 1 = not abusive.

MiniLM INT8

The small model that runs inside Chrome. “INT8” = compressed to 8-bit so it fits in a browser.

Recall vs specificity

Recall: share of abusive messages we catch. *Specificity*: share of clean messages we correctly leave alone.

p-value

Chance of seeing a difference this large if there were really no difference. Below 0.05 we call it real.

ONE SENTENCE, THE TWO FORMS THAT MATTER — both mean “there is a meeting at 4 tomorrow”

What our labelled data looks like: Devanagari Hindi कल मीटिंग है चार बजे

What a real user types: Roman Hinglish kal meeting hai 4 baje

PART I

Problem

Why an abuse detector that scores 84% can still be untested on real traffic.

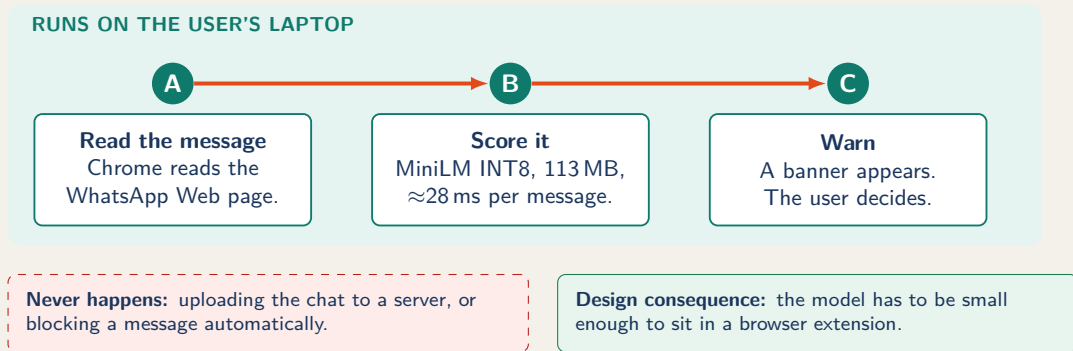
Questions this part answers

What ships today? A Chrome extension that scores WhatsApp Web messages entirely on the user's laptop.

What was measured? Hindi in Devanagari script, and English. Never the Roman Hinglish people actually type.

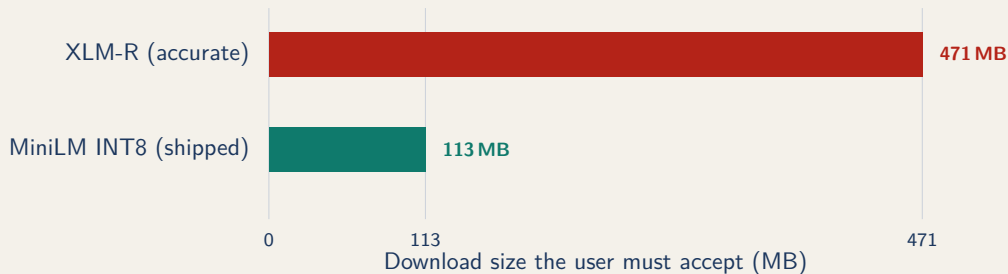
Why not just collect Hinglish data? No labelled Hinglish abuse dataset exists for this task.

What we built: a warning, not a filter, and it never uploads



Why this matters for everything that follows: every decision that follows is constrained by a model that must fit on a laptop and answer in milliseconds.

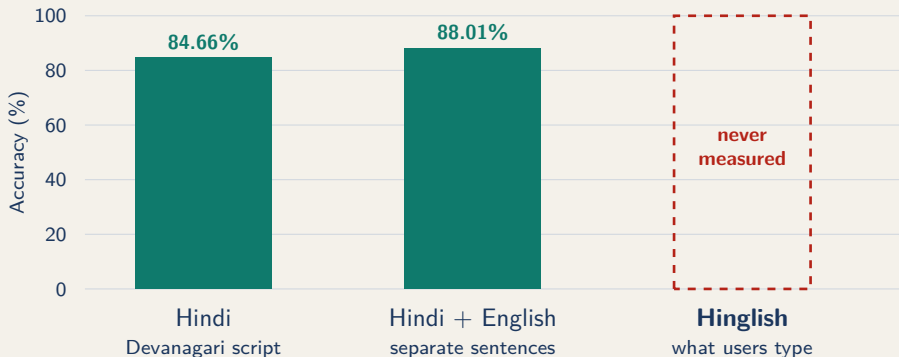
The accurate model is four times too large to ship



What shrinking costs us. On the published Hindi benchmark, XLM-R reaches 86.34% and our small model reaches 84.66% — a gap of under 0.11 pp once quantised.

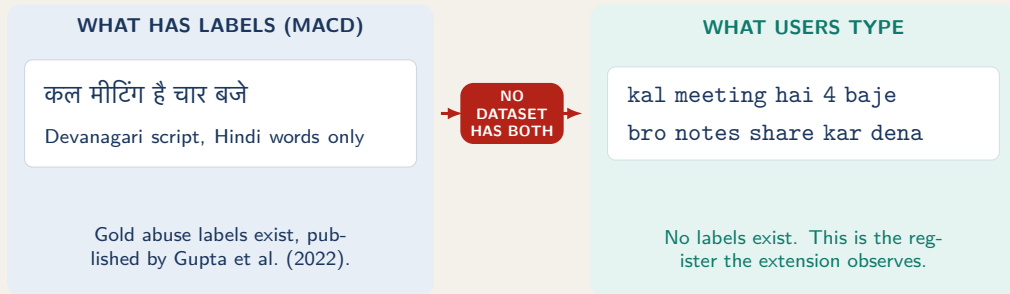
So the small model is not the problem. The problem, on the next slide, is which *language* we ever tested it on.

The gap: the language our users type was never tested



The dashed box is not a low score — it is a *missing experiment*. Our earlier poster reported the two solid bars and stopped there. A detector deployed on WhatsApp sees almost none of either: it sees `kal meeting hai 4 baje`, which is neither of the two things we tested.

Why we could not simply download Hinglish training data



And the standard trick does not apply. The usual way to build code-mixed text (WAC and PAC) needs each sentence paired with its English translation. MACD ships Hindi only, with no English pair, so those methods are unusable here. That is what forced us toward a language model.

The research question, and the three rules we set ourselves

Can we rewrite labelled Hindi abuse data into natural Hinglish, keep the original labels honest, and prove it on a test set frozen before training?

Rule 1 — don't touch the product

The extension's read-score-warn path stays exactly as it is.

Rule 2 — don't launder the labels

A message labelled abusive must still be abusive after rewriting.

Rule 3 — don't move the goalposts

The 6368-message Hinglish test set is frozen before any training.

Rule 2 is the one that later turns out to be hard to verify, and Part IV is mostly about that.

PART II

Method

Turning Devanagari Hindi into Hinglish without inventing new labels.

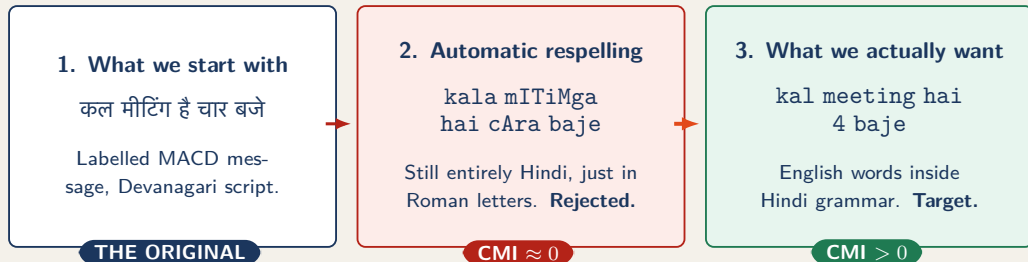
Questions this part answers

Can't we just respell it in Roman letters? We tried and measured it: respelling changes the script, not the language.

So how do we get Hinglish? An LLM rewrites each message once, offline. It never runs inside the extension.

How do we know the rewrite is still abusive? A two-part gate keeps or quarantines each rewrite. Part IV measures how well it does that.

First attempt: respelling Hindi in Roman letters is not mixing



Respelling was the cheap option — no GPU, no cost. But the code-mixing index stayed at zero, which is the measurement telling us the sentence is still monolingual Hindi. **Changing the alphabet is not changing the language**, so a model trained on column 2 would still never see an English word.

Our method: one offline rewrite pass, outside the product

WHAT THE USER RUNS — no language model here

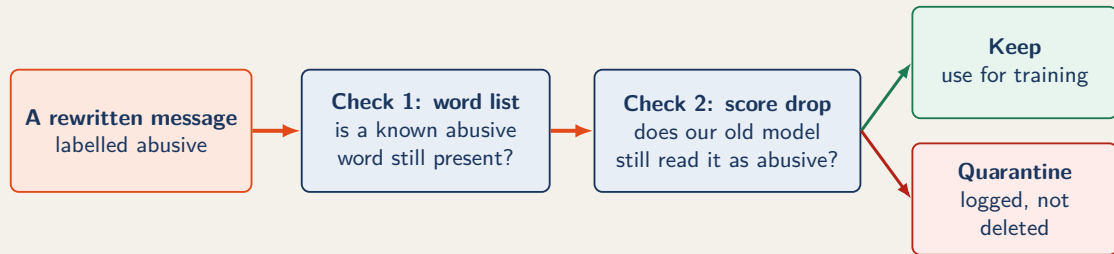


WHAT WE RAN ONCE, ON A SERVER — training data only



31 663 messages were rewritten once through Qwen3-32B on AWS Bedrock. The language model is a *data converter*, not a detector: it is never downloaded, never called at inference, and cannot affect a user's latency or privacy. Only its output — rewritten training rows — reaches the shipped model.

Keeping the labels honest: how the gate works



The worry: a language model asked to rewrite an insult might quietly *soften* it. The message would then sit in training data labelled “abusive” while no longer being abusive — and we would be teaching the detector something false.

Check 2 in plain terms: we score the Hindi original and the Hinglish rewrite with our previous detector. If the rewrite’s abuse score falls by more than 0.15, we assume abuse was lost and quarantine the row. Written out: $p_{\text{rewrite}} \geq p_{\text{original}} - 0.15$.

Freezing the test set before training, so the result means something

Allowed

- Cleaning or re-labelling the *training* rows.
- Quarantining a rewrite, as long as it is logged rather than deleted.
- Reporting the code-mixing index even when the value is unfavourable.

Forbidden

- Editing the 6 368 frozen test messages after seeing model errors.
- Scoring variants on anything but the identical frozen set.
- Putting the language model inside Chrome.

A test set chosen *after* looking at errors will flatter any model, and the number stops being comparable to the published baseline. So we fixed the 6 368 Hinglish messages first and did not touch them again.

Because every variant is scored on that same set, **Part IV** can compare them with a paired test.

PART III

Results

What the retrained detector scores, and how human-like the generated Hinglish is.

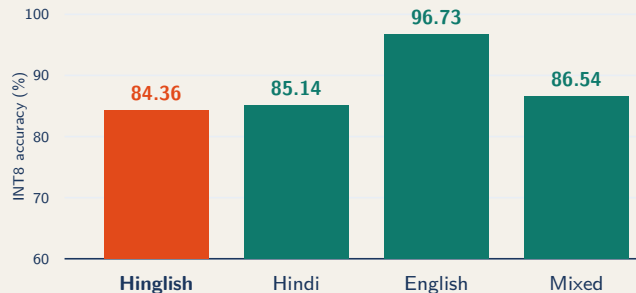
Questions this part answers

Does it work on Hinglish now? Yes — 84.36% on the frozen test set, with the extension untouched.

Does the Hindi capability survive? Yes, at 85.14%. An ablation showed why the final mix must keep the Hindi data.

How realistic is the generated Hinglish? Fluent, but it mixes more shallowly than human writing — the observation that drives Part IV.

Result: the missing cell is filled, at 84.36%



The product did not change.

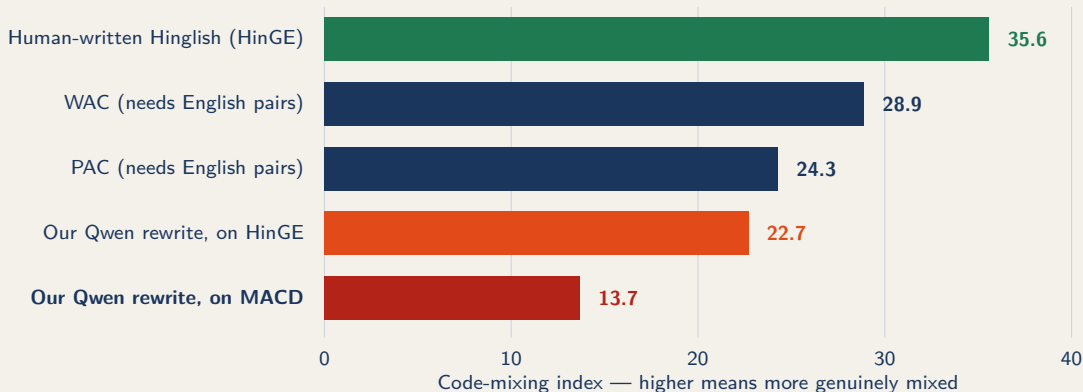
Same MiniLM, same 113 MB, same ≈ 28 ms. Only the training data changed.

Ablation behind this mix.

Training on Hinglish *in place of* Hindi gives Hindi accuracy of only **70.04%**. Keeping both sources re-stores it to 85.14%.

Frozen Hinglish test, $n = 6368$, never seen during training. “Mixed” is all three languages together; it reads 86.54% rather than the earlier 88.01% precisely because Hinglish is now inside the mix and Hinglish is the hardest of the four. **A Hinglish gain paid for with a Hindi loss is not a gain**, which is why the final recipe trains on both languages together.

Measuring the generated Hinglish against human writing



Every automatic method sits below human writing, and ours sits lowest on the corpus we train on: **13.7 against 35.6**, with **25%** of rewrites containing no mixing at all (CMI = 0). On translation-style quality our rewrites are competitive (BLEU 17.7 versus WAC 16.4 and PAC 17.9), so the gap is specifically in *depth of mixing*, not fluency. That distinction is what Part IV follows up on.

PART IV

Analysis

Depth of mixing is where our data differs most from real writing — so we measured what that costs.

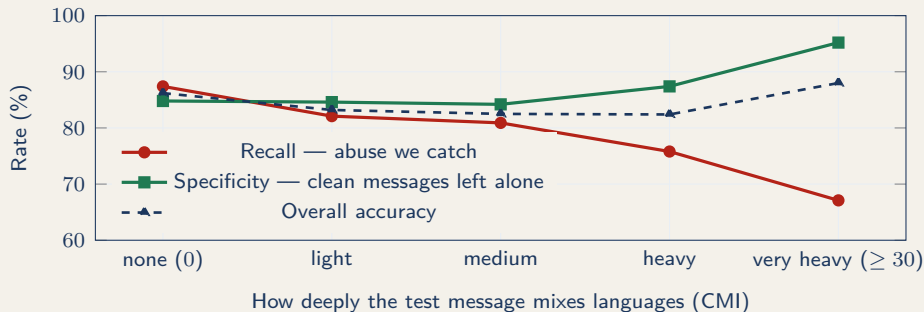
Questions this part answers

How does accuracy vary with mixing depth? We score the detector stratum by stratum instead of as one average.

How good is the label gate? We build a reference set and measure its accuracy directly, per stratum.

Are the design choices statistically supported? Every comparison is re-run as a paired test on identical messages.

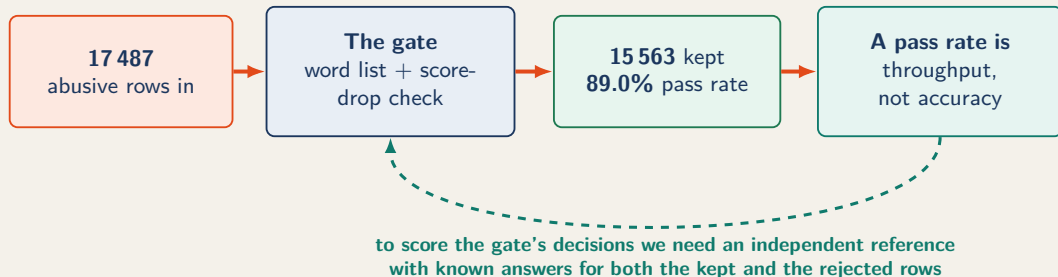
Scored by mixing depth: recall falls as mixing deepens



Read the red line. On unmixed messages the detector catches 87.4% of abuse; on the most heavily mixed it catches 67.1%. A trend test confirms the slope is systematic rather than noise ($p = 5.5 \times 10^{-13}$).

Why the average does not show it. Heavily mixed messages are mostly *clean* — the abusive share falls from 54.7% to 25.6% — so predicting “not abusive” is rewarded in that stratum and accuracy (dashed) even rises. For a safety feature, recall by depth is therefore the metric we track alongside accuracy from here on.

Next question: how accurate is the label gate itself?



Two details that shaped the study. Of the 1924 rejections the word list accounts for **74** and format checks for 44; the score-drop check accounts for **1 806**, so the gate's behaviour is dominated by the score check rather than the word list. And its 0.15 threshold was inherited from an earlier script rather than tuned for this task — so the threshold is treated as a free parameter and swept, not assumed.

Designing a reference set the gate can be scored against

Design 1 — rejected

Auto-build a slur list and delete one slur per message. Validation showed 24% of those rows still contained a slur, and harmless words such as kapde (clothes) were being deleted.



Design 2 — rejected

Hand-pick unambiguous slurs and verify each deletion. The labels are now sound, but deletion leaves broken grammar — a cue the gate could exploit without judging abuse at all.

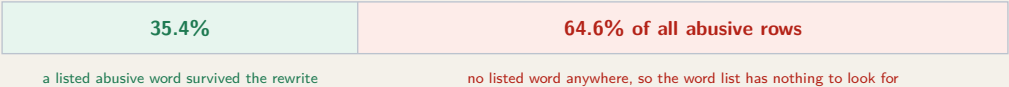


Design 3 — adopted

Ask the same model to rewrite each message twice: once faithfully, once deliberately softened. Both versions are fluent, so no checker can win by spotting bad grammar.

A checker cannot be scored without cases whose correct answer is known independently of the checker. The adopted design gives **466 graded rewrites** from 233 source messages, built with roughly 2 100 model calls. **Stated limitation:** the “softened” label comes from the instruction given to the model and is verified only by word match, so certifying a human-labelled sample is the first task of the next iteration.

Measured gate accuracy on the rows the word list cannot see



Checker, on the 64.6% stratum	Catches real softening	Wrongly rejects faithful rewrites
Word list	0.0%	0.0%
Score drop > 0.15 (current setting)	43.8%	24.9%
Score drop, best threshold	52.8%	32.2%
An LLM shown the original	27.0%	1.3%

All four rows are the same 466 graded rewrites, so they are directly comparable. The word list is not weak here — it simply has no term to match, which is the case on **64.6%** of the abusive corpus. That leaves the score check deciding alone at AUC **0.629** against 0.500 for chance, and a threshold sweep does not move it. **The design brief:** keep the word list on the 35.4% it covers, and put a stronger judge on the rest.

Re-testing every data decision as a paired significance test

Comparison	Current	Alternative	<i>p</i> -value	Conclusion
current mix vs extra data cleaning	84.28%	83.81%	0.179	not distinguishable
current mix vs cleaning + synthetic	84.28%	83.46%	0.027	significantly worse

Why the test was necessary

On $n = 6368$, a single accuracy carries a 95% interval of about ± 0.89 pp — wider than both gaps below. McNemar’s test scores the two models on the *same* messages and counts only where they disagree, which resolves differences that small.

What it changes

The synthetic-row variant is ruled out on evidence. The cleaning variant is reported as indistinguishable rather than as a win for either side, and stays an open question for the depth-stratified rerun.

Label noise, now expressible as a range. If one rewrite in five is softened, the measured catch rate puts mis-labelled rows at **15.8%** rather than 20%, at the cost of quarantining 1924 abusive rows. The softening rate itself is not yet measured, hence a range.

Limitations we carry into the next iteration

Selection on the test set

bounds 84.36%

The training script promotes the variant with the best *test* accuracy, so the headline should be read as an optimistic estimate. A held-out selection split is part of the rerun.

Model-derived labels

bounds the gate table

“Softened” comes from the instruction given to the model and is verified by word match, not by a human. Certifying a sample is task A of the next iteration.

Unmeasured softening rate

bounds 15.8%

We know what share of softening the gate catches, but not how often the rewriter softens in the first place — which is why label noise is quoted as a range.

None of these change the shipped product or the stratified measurements above. Each one bounds how strongly a single number may be stated — named against that number in orange — and each has a concrete fix on the next slide.

Conclusion: what this iteration established, and the next target

What this iteration established

- The product is unchanged and works: on-device, 113 MB, ≈ 28 ms.
- Hinglish went from unmeasured to measured, at 84.36%, with Hindi held at 85.14%.
- The binding constraint is mixing *depth*, not Hinglish itself.
- The label gate now has measured operating characteristics, per stratum.

Next iteration, in order

- A** Have humans label 400 rewrites and report agreement, so the reference set is certified independently of any model.
- B** Rebuild the gate per stratum: word list where it is near-perfect, an LLM judge on the rest.
- C** Generate training data balanced across mixing depth.
- D** Re-freeze a depth-stratified test set; use a paired test for every decision.

The target for the final report. Catch **82%** of abuse in heavily mixed messages, up from **72.4%** today, without letting clean-message accuracy fall below 84%.